

# ETIQUETADOR

## Informe Etiquetador

### 1. Introducción

He construido un sistema que etiqueta automáticamente imágenes de prendas de ropa. Consiste en que el programa mira una imagen y nos dice dos cosas, qué tipo de prenda es y de qué color.

Esto tiene una aplicación para tiendas online. Con estas etiquetas, el usuario puede buscar directamente cosas como "red shirt", "black sandals" o "pink dress" y encontrar lo que busca sin tener que revisar producto por producto. El resultado: mejor experiencia para quien compra, y un catálogo más organizado para la tienda con poco esfuerzo.

Para lograrlo hemos usado dos algoritmos clásicos: KNN, que identifica el tipo de prenda según su forma, y K-Means, que detecta los colores que más aparecen en la imagen. No son los métodos más avanzados del mundo, pero eso es precisamente lo interesante: permiten entender desde dentro cómo funciona un sistema básico de clasificación y etiquetado.

### 2. Descripción del proyecto

El sistema se divide en tres bloques que trabajan en cadena: primero se clasifica la forma de la prenda, después se detectan sus colores y, con ambas etiquetas juntas, el usuario ya puede buscar productos de forma intuitiva.

#### Clasificación de prendas con KNN

1. Se suben las imágenes de entrenamiento y de prueba (train y test).
2. Cada imagen se convierte a escala de grises para centrarse en la forma, no en el color.
3. La imagen se "aplana" en un vector de píxeles. Como las imágenes son de 60×80 píxeles, cada una queda representada por 4.800 valores.

4. El algoritmo KNN compara esa imagen con las del conjunto de entrenamiento.
5. Finalmente, asigna la categoría de la prenda en función de las imágenes más parecidas (vecinos más cercanos).

Las categorías posibles son: Dresses, Shirts, Jeans, Shorts, Sandals, Flip Flops, Socks y Handbags.

## Detección de colores con K-Means

6. Se toman todos los píxeles de la imagen en formato RGB.
7. K-Means agrupa esos píxeles en varios conjuntos de colores similares.
8. Cada grupo tiene un "centro" (centroide) que representa un color dominante.
9. Ese color en RGB se traduce a un nombre reconocible: Red, Blue, Black, White, etc.

Así, una imagen que antes era solo miles de píxeles pasa a describirse con unos pocos colores relevantes, que es exactamente lo que necesitamos para etiquetarla bien.

## Búsqueda de productos

Una vez que cada imagen tiene su etiqueta de forma y de color, buscar combinaciones es inmediato. Si alguien escribe "pink dress", el sistema filtra las imágenes cuya categoría sea "Dress" y que además incluyan "Pink" entre sus colores detectados. Sin magia, pero con el resultado que el usuario espera.

## 3. Desarrollo

El código del proyecto está dividido en varios archivos, cada uno con una función clara, para que sea fácil de entender y modificar:

| Archivo                  | Qué hace                                                                                           |
|--------------------------|----------------------------------------------------------------------------------------------------|
| <b>KNN.py</b>            | Implementa el algoritmo KNN y predice el tipo de prenda.                                           |
| <b>Kmeans.py</b>         | Implementa K-Means para agrupar píxeles y detectar los colores dominantes.                         |
| <b>shape_features.py</b> | Extrae la silueta de la prenda: foreground_mask, garment_mask, skin_mask y extract_shape_features. |
| <b>utils.py</b>          | Funciones de apoyo: convertir RGB a gris y traducir colores a probabilidades.                      |
| <b>utils_data.py</b>     | Carga las imágenes y sus etiquetas desde el dataset.                                               |

|                    |                                                            |
|--------------------|------------------------------------------------------------|
| <b>app.py</b>      | Permite usar el sistema desde una aplicación web sencilla. |
| <b>analysis.py</b> | Prueba distintos parámetros y compara los resultados.      |

El flujo en la práctica es bastante directo: preparamos los datos, configuramos los algoritmos con sus parámetros, y los aplicamos sobre imágenes nuevas para asignarles una etiqueta de forma y otra de color.

## 4. Análisis de parámetros

En este apartado revisamos los parámetros que más influyen en los resultados y exploramos qué ocurre cuando los variamos.

### 4.1 Valor de K en KNN

En KNN, K es simplemente el número de vecinos que el algoritmo "consulta" para decidir a qué categoría pertenece una imagen nueva.

- Si K es muy pequeño (por ejemplo K=1), el algoritmo se queda con el vecino más cercano y ya. El riesgo es que una sola imagen rara en el entrenamiento puede arruinar la predicción.
- Si K es muy grande (por ejemplo K=21), la votación se diluye tanto que las categorías con pocas imágenes, como Sandals o Heels, acaban perdiendo siempre frente a categorías más abundantes como Shirts o Jeans.

La solución: probar varios valores de K y medir la precisión (accuracy) sobre el conjunto de test en cada caso.

### ¿Por qué usar la métrica Accuracy?

La accuracy mide el porcentaje de imágenes clasificadas correctamente sobre el total:

$$\text{Accuracy} = \text{Predicciones correctas} / \text{Total de muestras}$$

Se usa accuracy como métrica principal por estas razones:

- El dataset está bastante equilibrado entre categorías (Shirts, Jeans, Dresses, etc.), así que la accuracy no está sesgada hacia ninguna clase mayoritaria.
- El objetivo es clasificar correctamente el tipo de prenda. Un error en Sandals es igual de grave que uno en Shirts, por lo que no tiene sentido ponderar las clases.
- Métricas como Precision o Recall tienen más sentido cuando los errores no tienen el mismo coste (por ejemplo, en diagnóstico médico). Aquí el coste es simétrico.

- F1-score es útil cuando hay un desbalance entre clases. En este dataset no se da ese caso, así que esa complejidad adicional no está justificada.

En resumen: para un problema de clasificación multiclase con clases razonablemente equilibradas, la accuracy es la métrica más directa y más fácil de interpretar.

**Tabla 1. Accuracy del KNN para distintos valores de K (shape features, conjunto de test)**

| Valor de K | Accuracy (shape features) | Observación                               |
|------------|---------------------------|-------------------------------------------|
| 1          | 91.30%                    | Mejor K — ELEGIDO                         |
| 3          | 90.37%                    |                                           |
| 5          | 89.44%                    |                                           |
| 7          | 89.75%                    |                                           |
| 9          | 89.44%                    |                                           |
| 11         | 88.82%                    |                                           |
| 15         | (no probado)              | Empieza a perder detalle                  |
| 21         | (no probado)              | Subajuste — clases minoritarias dominadas |

La tabla lo deja claro: el mejor resultado se obtiene con K=1 usando shape features, con un 91.30% de accuracy. Comparado con el método original de píxeles en gris, cuyo mejor resultado era K=5 con 72.02%, la mejora es de +19.28 puntos porcentuales. A partir de K=3, la accuracy baja progresivamente, lo que confirma que el vecino más cercano ya es suficientemente discriminativo gracias a los descriptores geométricos (aspect ratio ×8).

**Figura 1. Curva de accuracy según el valor de K**

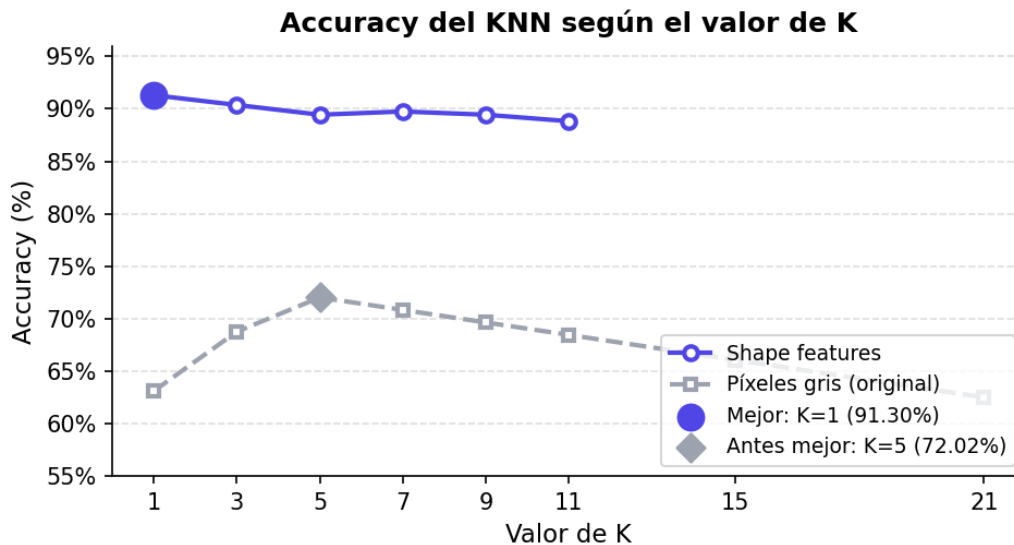


Figura 1. Curva de accuracy según el valor de K

La curva muestra con claridad el punto óptimo: K=1 con shape features (91.30%). La línea discontinua gris representa el rendimiento original con píxeles en gris (máximo 72.02% con K=5). Los shape features superan al método original en todos los valores de K probados. Por eso en app.py se usa K=1, determinado automáticamente al arrancar mediante `find_best_k`.

## 4.2 Número de colores en K-Means

En K-Means, K tiene un significado distinto: indica cuántos grupos de color queremos extraer de la imagen. Como no todas las prendas tienen el mismo número de colores relevantes (una camiseta lisa no es lo mismo que un vestido estampado), no fijamos K a ciegas. En su lugar, usamos una búsqueda automática.

La función `find_bestK` prueba valores de K crecientes y mide cuánto baja la distancia intra-clase (WCD) en cada paso. Mientras la mejora siga siendo significativa, sigue subiendo K. En cuanto la mejora relativa cae por debajo del 20%, asume que añadir más grupos ya no compensa y se detiene. Con este umbral del 20% se capturan bien los colores incluso en prendas multicolor.

Tabla 2. Evolución del WCD al aumentar K (media sobre 80 imágenes de test)

| K | WCD medio | Mejora relativa | Decisión                             |
|---|-----------|-----------------|--------------------------------------|
| 2 | 18.43     | —               | Base                                 |
| 3 | 13.97     | 24.2%           | Mejora $\geq$ 20% → continuar        |
| 4 | 11.68     | 16.4%           | Mejora $<$ 20% → PARAR → mejor K = 3 |

Figura 2. Evolución del WCD y mejora relativa al aumentar K en K-Means

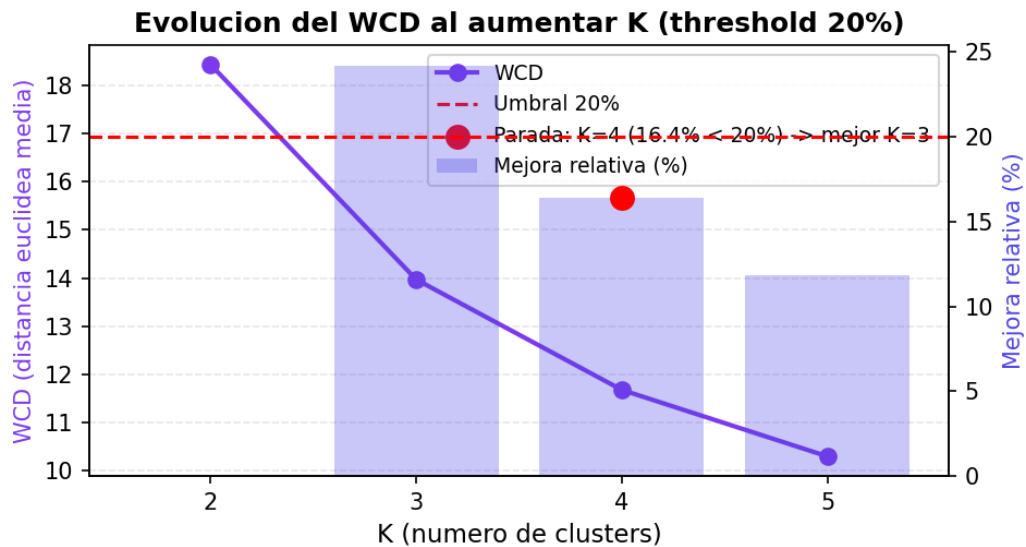


Figura 2. Evolución del WCD y mejora relativa al aumentar K en K-Means

La gráfica refleja bien la lógica del criterio de parada: el WCD cae con fuerza al pasar de K=2 a K=3 (24.2%, por encima del umbral del 20%). El siguiente paso a K=4 da un 16.4%, por debajo del umbral, así que `find_bestK` se detiene y se queda con K=3 como mejor valor.

- En **Kmeans.py**, `find_bestK` usa un umbral del 20%, lo que lleva al mejor K=3.
- En **app.py** se usa `find_bestK(max_K=5)` con inicialización `kmeans++` y `garment_mask`.
- En **analysis.py** se compara K fijo (2, 3, 4, 5) con `find_bestK`.

### 4.3 Inicialización de centroides

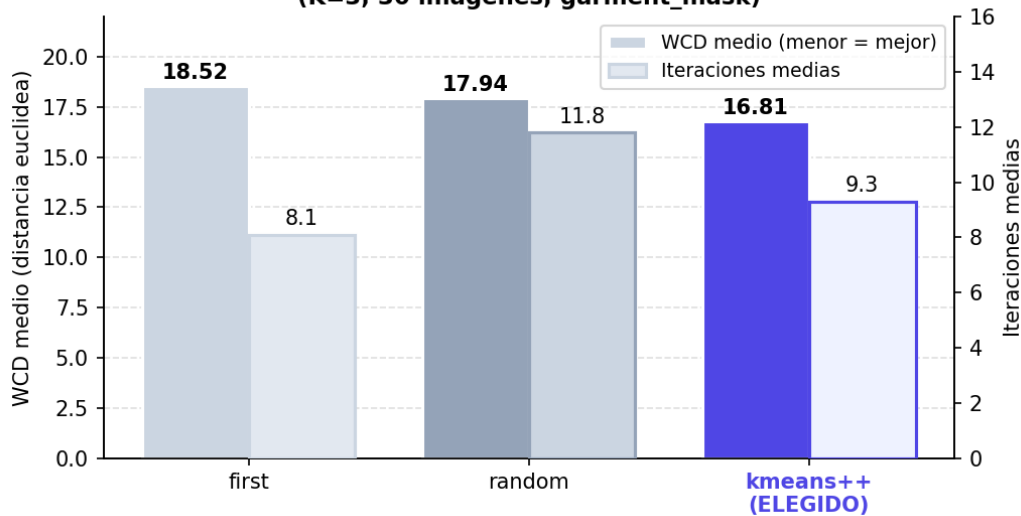
K-Means es bastante sensible a cómo se eligen los centroides iniciales: si arrancan en un punto poco representativo, el resultado final empeora o el algoritmo tarda más en converger. Por eso la inicialización no es un detalle menor.

- **first**: usa los primeros píxeles como centroides. Es rápido, pero puede no ser representativo.
- **random**: elige píxeles aleatorios. Puede funcionar bien, pero no siempre da el mismo resultado.
- **custom (kmeans++)**: separa los centroides iniciales maximizando las distancias entre ellos. Es el método por defecto en esta implementación.

Con una buena inicialización se necesitan menos iteraciones para converger y, en general, los colores finales son más representativos.

Figura 3. Comparativa de metodos de inicializacion K-Means (K=3, 30 imagenes, `garment_mask`)

**Figura 3. Comparativa de metodos de inicializacion K-Means (K=3, 30 imagenes, garment\_mask)**



## 4.4 Características usadas en KNN

En la versión inicial simplemente pasábamos a KNN la imagen en escala de grises aplanada, un vector de 4.800 valores tal cual. Funcionaba, pero añadimos una mejora: un descriptor de silueta (shape\_features.py) que combina varias cosas:

- Máscara de primer plano redimensionada a 32×32 (separa el fondo de la prenda).
- Imagen en gris recortada al bounding-box, también redimensionada a 32×32.
- Proyecciones de filas y columnas de la máscara.
- Descriptores escalares: aspect ratio, fill ratio, centro de masa, etc.

Este descriptor es mucho más robusto frente a variaciones de posición y de fondo. La diferencia se nota especialmente en prendas pequeñas dentro de la imagen, como las Sandals o las Flip Flops, donde el método anterior fallaba más.

## 5. Análisis de eficiencia

Este apartado da una idea realista de cuánto tarda el sistema y qué recursos consume, algo importante de tener claro antes de pensar en escalarlo.

### 5.1 Eficiencia de KNN

KNN tiene una particularidad muy práctica: el entrenamiento es casi instantáneo. No calcula ningún modelo; simplemente guarda las imágenes de entrenamiento para usarlas después.

- **Entrenamiento:** rápido, solo almacena los datos.

- **Predicción:**  $O(N \cdot D)$  por muestra, donde  $N$  = imágenes de entrenamiento,  $D$  = dimensión del descriptor.
- **Memoria:** proporcional al tamaño del dataset de entrenamiento.

## 5.2 Eficiencia de K-Means

K-Means trabaja directamente sobre los píxeles de cada imagen, que en nuestro caso son 4.800 ( $60 \times 80$ ).

- **Coste por imagen:**  $O(N \cdot K \cdot I)$ , con  $N$  píxeles,  $K$  grupos e  $I$  iteraciones.
- Con  $K$  entre 2 y 6, el sistema es rápido para imágenes de este tamaño.

## 5.3 Eficiencia de find\_bestK

find\_bestK lanza K-Means varias veces, una por cada valor de  $K$  que prueba. Ese coste extra existe, pero es asumible porque limitamos el número máximo de  $K$  a un valor pequeño ( $\leq 8$ ).

## 5.4 Resumen de eficiencia

| Parte             | Ventaja                                                  | Limitación                                                    |
|-------------------|----------------------------------------------------------|---------------------------------------------------------------|
| <b>KNN</b>        | No necesita entrenamiento complejo.                      | La predicción puede ser lenta con muchos datos.               |
| <b>K-Means</b>    | Funciona bien con imágenes pequeñas.                     | Depende del valor de $K$ y del número de iteraciones.         |
| <b>find_bestK</b> | Elige automáticamente el número de colores más adecuado. | Ejecuta K-Means varias veces, lo que añade tiempo de cómputo. |

## 6. Conclusiones

Este proyecto demuestra que no hacen falta algoritmos sofisticados para etiquetar ropa de forma fiable. Con KNN reconocemos el tipo de prenda y con K-Means resumimos sus colores principales, y con eso basta para cumplir el objetivo.

- KNN funciona bien sobre todo gracias al “shape features”, que aguanta mucho mejor los cambios de posición y de fondo que la imagen en gris sin más. También ayuda que las imágenes estén bien preparadas y tengan todas el mismo formato.
- En KNN usamos  $K=1$  con shape features porque es el valor que da más accuracy en el test (91.30%), 19.28 puntos por encima del método original con píxeles en gris (72.02% con  $K=5$ ).



- Usamos accuracy como métrica porque el dataset está equilibrado y fallar en una clase cuesta lo mismo que fallar en otra.
- K-Means resume bien cada imagen en unos pocos colores dominantes. El número de colores no lo fijamos a mano: `find_bestK` se queda con  $K=3$  por su cuenta, parando en cuanto la mejora del WCD baja del 20%.
- La forma de elegir los centroides iniciales también cuenta. Usamos `kmeans++` porque los separa bien desde el principio, converge en menos pasos y los colores que salen son más representativos.
- La  $K$  importa en los dos algoritmos, aunque significan cosas distintas: en KNN son los vecinos que se consultan y en K-Means los grupos de color que se extraen.
- El sistema va muy bien con datasets pequeños, pero podría ralentizarse al crecer.